NPOS
Non Preemptive Operating System
Architektura oprogramowania

Adrian SZCZEPAŃSKI

ABSTRAKT

NPOS jest kooperatywnym systemem przeznaczonym na urządzenia embedded. Powstał na potrzeby urządzeń wbudowanych typu 'safety-critical', gdzie zastosowanie systemu RTOS z wywłaszczeniami jest niemożliwe. W systemie oprócz mechanizmu podziału czasu między wątkami opracowany został kompletny mechanizm komunikacji międzyprocesowej (IPC). W systmie zastosowano istotny rozdział pomiędzy podziałem czasu procesora (task), od logiki obsługi wiadomości międzyprocesowej (service). Ponadto oprogramowanie zawiera interfejs do komunikacji z systemem plików, urządzeniami peryferyjnymi oraz konsolę systemową. Niniejszy dokument zawiera opis najważniejszych elementów systemu jak również zasady tworzenia własnych aplikacji na ten system.

2 November 2026

1. Scheduler

Moduł **nps** biblioteki **npos** (katalog `npos/libnpos/nps`) zawiera planistę zadań (*Scheduler*) Jest to planista kooperacyjny: zadanie wykonuje jedną porcję pracy w metodzie `process()` i oddaje sterowanie. Nie ma wywłaszczania zadań. Przestrzeń pamięci taska jest wspólna dla całego systemu. Oczywiście przerwania urządzenia nadal przerywają działanie taska, ale ich pełna obsługa powinna być obsłużona w systemie (więcej w rodziale `Urządzenia`)

Scheduler może mieć swojego następcę (*Successor*) , który wykonuje zadanie w momencie, gdy bieżący planista nie ma gotowych zadań. Mechanizm powstał na potrzeby mechanizmu ,*ARM*Trustzone-M aby system mógł pracować w przestrzeni Non-secure i Secure jednocześnie. Scheduler, na którym wywołana jest metoda `start()` jest nadrzędny w stosunku do swojego następcy, co oznacza, że dopóki ma gotowe zadania do wykonania, następca nie będzie uruchamiany.

Do schedulera przypisać można dwa zadania specjalne:

- **idle** - Zadanie wykonywane, gdy planista, ani następca nie mają gotowych zadań. Służy do wprowdzania urządzenia w stan niskiego poboru energii.
- **watchdog** - Zadanie wykonywane do odświeżania sprzętowego watchdoga, w celu zapobiegania jego zawieszeniu.

Najważniejsze metody schedulera to:

`Scheduler(TaskListInt&`taskList) - konstruktor. Jako argument przyjmuje listę zadań do obsługi przez planistę w postaci wektora wskaźników na taski. Lista ta powinna być pusta, scheduler sam zarządza dodawaniem i usuwaniem zadań.

`addTask(Task&`task) - Dodaje zadanie do listy zadań. Jeżeli lista jest pełna zwraca *false*.

`start()` - inicalizuje taski, uruchamia scheduler w nieskończonej pętli.

`setSuccesor(SchedulerSuccessor&`ss) - Ustawia następce planisty. Może to być kolejny obiekt schedulera lub np. proxy do planisty w innej przestrzeni ARM Truszone-M.

`setIdleTask(Task&`task) - Ustawia zadanie idle. Task nie jest zaliczany do puli tasków w systemie.

`setWatchdogTask(Task&`task) - Ustawia zadanie watchdog. Task nie jest zaliczany do puli tasków w systemie.

`stop()` - Zatrzymuje scheduler.

1.1. Task

Task jest pojedynczym zadaniem w systemie, określającym jednostkę pracy do wykonania przez scheduler. Kolejność uruchamiania tasków określa priorytet przypisany do każdego taska. Im większa liczba, tym wyższy priorytet i zadanie uruchomione zostanie wcześniej.

Najważniejsze metody taska to:

`initialize()` - Metoda wykonywana przez scheduler przy starcie.

`isReady()` - Zgłasza gotowość taska do wykonania. Metoda powinna zwracać *true* jeżeli task jest gotowy do przetworzenia, w przeciwnym razie *false*.

`process()` - Metoda wykonywana przez scheduler w celu realizacji jednostki pracy przypisanej do taska.

2. Komunikacja międzyprocesowa

Moduł **ipc** biblioteki **npos** (katalog `npos/libnpos/ipc`) definiuje wiadomości, adresowanie oraz magistralę łączącą zadania.

W systemie zastosowano rozdzielenie mechanizmu podziału czasu (*task*) od mechanizmu komunikacji międzyprocesowej (*service*) Oznacza to, że użytkownik może niezależnie tworzyć obsługę wiadomości, bez konieczności ingerencji w mechanizm podziału czasu i kolejkowania wiadomości. Na etapie konstruowania konkretnej instancji systemu należy jedynie przypisać serwis do obiektu taska, który będzie go obsługiwał (więcej o tym w dalszej części rozdziału).

2.1. Adresowanie

Podstawowym elementem odniesienia w systemie komunikacji międzyprocesowej jest identyfikator obiektu (OID - *Object Identifier*). OID określa zarówno serwis użytkownika, uchwyt do pliku czy urządzenie w systemie. OID składa się z:

`portId` - Identyfikator portu, do którego trafia wiadomość na magistrali. Portem jest np. task z przypisanymi serwisami, czy task zarządzający plikami w systemie (o portach będzie w dalszej części rozdziału).

`serviceId` - Identyfikator obiektu w ramach portu. Obiektem może być np. serwis użytkownika, plik itp.

Specjalnym przypadkiem jest `NULL_OID`, który oznacza brak przypisanego obiektu w systemie. Najczęściej jest używany do rozgłaszania wiadomości (*broadcast*), gdy odbiorca nie jest znany serwisowi.

2.2. Wiadomości

W pliku `Message.hpp` zdefiniowano strukturę wiadomości. Składa się z nagłówka oraz struktur *Input* i *Output*.

Header składa się z:

`Oid sender` - nadawca wiadomości.

`Oid recipient` - odbiorca wiadomości.

`ReferenceCount referenceCount` - licznik referencji wiadomości.

`Type type` - typ wiadomości.

WAZNE! Użytkownik nie powinien modyfikować pól nagłówka wiadomości, gdyż są one zarządzane przez system.

Struktury *Input* oraz *Output* zawierają dane przesyłane w wiadomości. Zawierają one pola specyficzne dla danego typu wiadomości oraz pole `raw` dla niestandardowych danych definiowanych przez użytkownika. Jeden obiekt wiadomości wykorzystywany jest zarówno do wysyłania żądania jak i odpowiedzi na nie. Zasada jest taka, że serwis nadawczy wypełnia pola ze struktury *Input*, a odbiorca przetwarza dane i wypełnia pola ze struktury *Output* (nie dotyczy przypadków odczytu *read*, gdzie nadawca wypełnia pola `data` i `size` w

strukturze *Output*).

WAZNE! Pola `data` w strukturach *Input* i *Output* zawierają wskaźnik do danych, ale nie zapewniają automatycznego zarządzania pamięcią. Użytkownik musi samodzielnie zapewnić przydział pamięci do danych do czasu otrzymania odpowiedzi!.

2.2.1. Typy wiadomości

Typ wiadomości jest podstawowym mechanizmem rozróżniania wiadomości w systemie.

WAZNE! Stosowanie tego samego typu dla różnych wiadomości może prowadzić do konfliktów w systemie!

W pliku `Message.hpp` zdefiniowano zakresy typów wiadomości:

```
OS_ERROR_MESSAGE_TYPE = 0x0000;
DEVICE_MESSAGE_TYPE = 0x0100;
DRIVER_MESSAGE_TYPE = 0x0200;
FILESYSTEM_MESSAGE_TYPE = 0x0300;
CLI_MESSAGE_TYPE = 0x0400;
USER_MESSAGE_TYPE = 0x0500;
```

Użytkownik powinien definiować własne typy wiadomości w zakresie od `USER_MESSAGE_TYPE` do `UINT16_MAX` (0xFFFF) i unikać konfliktów z typami zdefiniowanymi w systemie.

2.2.2. Status wiadomości

Odbiorca wiadomości powinien uzupełnić pole `status` w strukturze *Output*. Status wiadomości informuje nadawcę o wyniku przetwarzania wiadomości przez odbiorcę.

W strukturze `Message` zdefiniowano statusy zwracane automatycznie przez system:

```
STATUS_MESSAGE_NOT_SUPPORTED = 0xFF;
STATUS_PORT_NOT_FOUND = 0xFE;
STATUS_SERVICE_NOT_FOUND = 0xFD;
```

Użytkownik definiując własne statusy wiadomości powinien unikać konfliktów z statusami zdefiniowanymi w systemie. Najlepszą praktyką jest stosowanie typu `enum` indeksowanego od zera. W systemie przyjęto założenie, że status o wartości `0`(zero) oznacza brak błędów.

2.2.3. Definiowanie własnych wiadomości

Własne typy wiadomości powinny być strukturą dziedziczącą po strukturze `Message`. **NIE WOLNO JEDNAK ROZSZERZAĆ STRUKTURY O DODATKOWE POLA!**. Własne typy danych powinny być dopasowane do instniejącej struktury, a pobieranie i ustawianie wartości odbywać się powinno za pomocą przygotowanych funkcji dostępowych (getterów i setterów).

Przykłady definicji własnych wiadomości znajdują się w folderach `libnpos/dev/msg`, `libnpos/fs/msg`, czy `libnpos/cli/msg`.

2.3. Serwisy

Serwisy w systemie NPOS są elementem logiki tworzonej aplikacji. Są odpowiedzialne za obsługę wiadomości przychodzących i realizację odpowiednich działań w systemie.

Komunikacja z systemem odbywa się za pomocą interfejsu `Service::Api`, który dostarczany jest w konstruktorze obiektu serwisu. Gdy serwis chce wysłać żądanie musi wywołać metodę `send()`, natomiast gdy odpowiada na nadchodzące żądanie, powinien wywołać metodę `respond()`.

WAZNE! Odpowiadając na istniejącą wiadomość metodą `send()` zamiast `respond()` może spowodować nieprawidłowe działanie systemu!

Użytkownik tworząc serwis musi dostarczyć następujące metody:

`initialize()` - metoda uruchamiana podczas inicjalizacji serwisu. Może posłużyć do przygotowania zasobów lub konfiguracji serwisu, otwarcia pliku lub urządzeń.

`handleRequest(Message&)` - obsługa przychodzących żądań do serwisu. Żądanie może zostać obsłużone od razu lub odłożone na później (przez klasę `MessageStore`) w celu wysłania żądania do innego serwisu (np. odczytania zawartości pliku).

`handleResponse(Message&)` - obsługa odpowiedzi na wysłane wcześniej żądania.

`acceptsRequest(Message::Type) const` - metoda zwraca typ obsługiwanych żądań przez serwis. Nieobsługiwane typy zostaną automatycznie obsłużone przez system.

WAZNE! Nie należy samemu wywoływać metody `setId`, gdyż skutkuje to błędnym adresowaniem OID serwisu.

Przykłady implementacji serwisów znajdują się w folderach `libnpos/dev/service`, `libnpos/fs/service`, `libnpos/cli/service` lub w przykładach w folderze `examples`.

2.3.1. Maszyny stanów

Gdy obsługa żądania przez serwis wymaga wysłania kolejnych wiadomości lub oczekiwania na odpowiedzi, można wykorzystać klasę `FsmService`, który pozwala zaimplementować serwis w formie maszyny stanów.

Główną różnicą w stosunku do implementacji standardowego serwisu jest to, że w tym przypadku korzysta się z interfejsu klasy `FsmService::State`, który reprezentuje pojedynczy stan. Metody `handleRequest`, `handleResponse` zwracają `FsmService::State::ID`, który jest unikalnym identyfikatorem w maszynie stanów. W przypadku gdy obsługa wiadomości wymaga przejścia do innego stanu, metoda powinna zwrócić identyfikator tego stanu. W

przeciwnym wypadku powinna zwrócić `NO_STATE_CHANGE`, lub `getId()`.

W konstruktorze `FsmService` należy podać tablicę stanów w kolejności odpowiadającej ich ID. Także ID powinny być typem wyliczeniowym `enum` **indeksowanym od ZERA!**

2.4. ServiceTask

`ServiceTask` jest specjalnym rodzajem taska odpowiedzialnym za komunikację z magistralą systemową, odbiorem i kolejkowaniem wiadomości oraz przekazywaniem ich do serwisów. Jest także implementacją interfejsu `Service::Api`, , tzn. dostarcza mechanizmów do komunikacji z magistralą systemową poprzez serwisy.

Oprócz listy serwisów, `ServiceListInt` , `ServiceTask` zawiera też kolejki wiadomości przychodzących i wychodzących: `RequestQueueInt` oraz `ResponseQueueInt`. W kolejce nie są jednak przechowywane kopie wiadomości, a jedynie wskaźniki do nich, dzięki mechanizmowi alokacji w magistrali systemowej (więcej o tym w dalszej części rozdziału). Dzięki temu kolejki mogą być odpowiednio duże bez znacznego obciążenia pamięci (jeden slot kolejki zajmuje tylko 4 bajty w architekrurze 32-bitowej).

Wiadomości wysyłane między serwisami w ramach tego samego taska są wykonywane bezpośrednio, bez przechodzenia przez magistralę systemową i scheduler. Dzięki temu skraca się czas wykonania, gdyż wiadomośc nie przechodzi niepotrzebnie przez magistralę, wydłużając drogę wiadomości i alokując dodatkowo pamięć w globalnej puli. Z drugiej jednak strony jeżeli wiele serwisów wymienia wiadomości w ramach jednego taska, zwiększa się porcja czasu działania taska w ramach jednego cyklu schedulera, a więc gdy pojawi się zadanie o wyższym priorytecie, wydłuży się czas jego oczekiwania. Dotyczy to też resetowania licznika watchdog przez scheduler, co może prowadzić do zresetowania procesora. Dodatkowo zwiększa się stos wywołań, co przy ograniczonej pamięci urządzenia może prowadzić do jej przepełnienia. **Należy to wziąć pod uwagę przy projektowaniu własnej aplikacji!**

Najważniejsze metody to:

`ServiceTask(Priority priority, ServiceListInt& services, RequestQueueInt& requestQueue, ResponseQueueInt& responseQueue, Bus& bus)` - konstruktor. Oprócz priorytetu taska należy mu także dostarczyć kontener `ServiceListInt` do przechowywania listy serwisów. Nie należy go uzupełniać samemu, ponieważ task sam zarządza dodawaniem i przydzielaniem OID. Dodatkowo task wymaga dostarczenia kolejek wiadomości przychodzących i wychodzących.

`subscribe(Service& service)` - metoda na przyłączenie serwisu do taska. Po wywołaniu tej metody serwis otrzymuje przypisany OID.

2.5. Magistrala systemowa

Implementacja magistrali systemowej jest klasa `SystemBus`. Istotnym mechanizmem magistrali jest globalna pula do alokacji wiadomości trafiających na magistralę. Mechanizm ten pozwala na optymalizację zużycia pamięci, gdyż porty magistrali przechowują jedynie wskaźniki na wiadomości. Dzięki temu wyeliminowany został kosztowny proces kopiowania wiadomości między portami. Dodatkowo w przypadku wiadomości rozgłoszeniowych wszystkie porty odbiorcze operują na tym samym egzemplarzu wiadomości.

Mechanizm globalnej alokacji wymusza stosowanie jednak pewnych zasad przy obsłudze wiadomości. Przykładowo, w module `dev` zdefiniowana jest wiadomość `Open` służąca do odkrywania OID urządzeń. W przypadku gdy każde z urządzeń obsługuje ten typ wiadomości, wymagany jest dodatkowy mechanizm rozróżniający czy dana wiadomość adresowana jest do danego urządzenia (w tym przypadku poprzez pole *name*). Gdyby wiele urządzeń odpowiadałoby na tą samą wiadomość w kolejce odbiorczej taska pojawiłoby się wiele wskaźników do tej samej wiadomości. Ponadto wartość struktury `Output` wiadomości zawierałaby dane dotyczące ostatniego urządzenia, które odpowiedziało na tę wiadomość. **Należy to wziąć pod uwagę przy projektowaniu własnych wiadomości!**

Obiekty klasy `Message` zawierają pole `ReferenceCount`, które jest licznikiem odwołań do danej wiadomości. Magistrala zwiększa go za każdym razem, gdy port odbiera wiadomość. Rolą portu jest natomiast jego zmniejszanie po obsłużeniu wiadomości. Gdy licznik odwołań osiągnie zero, wiadomość zostanie zwolniona z globalnej puli alokacji.

> **WAZNE!** Z tego powodu użytkownik nigdy nie powinien sam modyfikować licznika `ReferenceCount`. W przypadku gdy obsługa wiadomości wymaga odłożenia jej na później, należy skorzystać z obiektu `MessageStore!`

Najważniejsze metody:

`SystemBus(PortListInt& ports, PoolInt& pool, PortId initialPort)` - konstruktor. Należy podać kontener do przechowywania wskaźników do portów, pulę alokacji wiadomości oraz identyfikator startowy portu.

`addPort(Port& port)` - dodaje port do magistrali oraz nadaje mu identyfikator.

`setSuccesor(SystemBus& succ)` - ustawia następcę magistrali systemowej, do którego będą przekazywane wiadomości w innym kontekście.

2.5.1. Porty i adresowanie

Do magistrali systemowej rejestrują się porty. Są to w większości obiekty `ServiceTask`, a także inne taski specjalne jak np. task systemu plików. Należy pamiętać, że mechanizm subskrypcji do schedulera jest niezależny od mechanizmu subskrypcji do magistrali systemowej.

Tak jak w przypadku klasy `nps::Scheduler` magistrala systemowa może mieć ustawionego następcę (`Succesor`), który służy do przekazywania wiadomości do schedulera w innym kontekście np. części Non-secure w ramach mechanizmu ARM Trustzone-M. Aby uniknąć konfliktu adresacji OID, w konstruktorze `SystemBus` podaje się identyfikator

startowy portu, od której magistrala ma adresować kolejne porty. Dzięki temu np. id portów w części Secure mogą zaczynać się od zera, a w części Non-secure od 128.

2.6. Przechowywanie wiadomości

W przypadku gdy obsługa wiadomości wymaga jej odłożenia na później (np. w celu wcześniejszego odczytania zawartości pliku lub skorzystania z urządzenia) należy skorzystać z obiektu `MessageStore`. W przypadku wiadomości pochodzących z magistrali systemowej obiekt ten zwiększy licznik odwołań. `MessageStore` obsługuje także żądania pochodzące z tego samego taska, poprzez skopiowanie wiadomości do lokalnego bufora.

Najważniejsze metody:

`MessageStore(Service& owner)` - konstruktor, należy podać referencję do właściciela magazynu wiadomości (serwisu).

`store(T& message)` - zapisuje wiadomość w magazynie wiadomości i zwraca wskaźnik do niej.

`release()` - zwalnia przechowywaną wiadomość, zmniejszając licznik odwołań i zwraca wskaźnik do niej.

`hasValue() const` - sprawdza, czy w magazynie znajduje się jakaś wiadomość.

`get() const` - zwraca wskaźnik do przechowywanej wiadomości, nie zmieniając licznika odwołań.

3. Urządzenia

Moduł **dev** biblioteki **npos** (katalog `libnpos/dev`) udostępnia urządzenia sprzętowe jako serwisy. Każde urządzenie ma interfejs warstwy sprzętowej (HAL) w `hal/`, wiadomości w `msg/`, serwis w `service/` oraz, dla zasobów systemowych, zadania w `task/`.

Zaletą tego rozwiązania jest przenaszalność aplikacji pomiędzy różnymi platformami sprzętowymi. Serwisy użytkownika komunikują się ze sprzęteme poprzez zdefiniowane wiadomości w module `dev::msg`. Również w przypadku, gdy interfejs HAL jest niekompatybilny z danym układem, użytkownik może opracować własne interfejsy i serwisy do ich obsługi. Ważne jest natomiast, aby zachować zgodność z mechanizmami komunikacji i zarządzania wiadomościami w module `dev`.

WAZNE! W przypadku definiowania własnych wiadomości do obsługi peryferiów, które nie są zdefiniowane w module `dev::msg` należy korzystać z typów w zakresie `USER_MESSAGE_TYPE` i unikać konfliktów z typami zdefiniowanymi w systemie.

3.1. Otwieranie urządzeń

Urządzenie jest identyfikowane nazwą. Klient wysyła rozgłoszeniowe `dev::msg::Open` (funkcja `dev::api::open()`). Odpowiada serwis o pasującej nazwie, przekazując w `getOpened()` swój Oid. Dalsze żądania są kierowane już bezpośrednio do tego Oid, dzięki czemu nie obciążają magistrali rozgłoszeniami.

3.2. Obsługa urządzeń

W pliku `libnpos/dev/Api.hpp` zdefiniowane są funkcje do obsługi urządzeń sprzętowych. Autor rekomenduje wywoływanie funkcji API do wysyłania żądań do sprzętu, co pozwoli na ewentualną zgodność aplikacji użytkownika w przypadku aktualizacji wiadomości w module `dev::msg`.

Przetwarzanie odpowiedzi wymaga sprawdzenia typu wiadomości (`ipc::Message::Type`), rzutowania jej na odpowiednią strukturę modułu `dev::msg`, a następnie skorzystania z getterów i setterów udostępnionych przez tą strukturę wiadomości. Istotnym elementem jest sprawdzenie statusu wiadomości, gdyż serwisy urządzeń jedynie zgłaszają błędy, ale to po stronie użytkownika spoczywa odpowiedzialność za ich obsłużenie w ramach pisanej aplikacji.

3.3. Obsługa przerwań

W pliku `libnpos/dev//hal/Interrupt.hpp` zdefiniowana jest struktura przerwania. Interfejs `Interruptable` służy do wyłączania przerwań na danym peryferium. Z drugiej strony interfejs `InterruptHandler` umożliwia obsługę przerwań generowanych przez peryferium. Przyjęto założenie, że jedno peryferium generuje przerwania dla jednego handlera.

Implementacją interfejsu `InterruptHandler` jest `InterruptService`. Jest to klasa dziedzicząca po zwykłym serwisie, jednak dodatkowo posiada kolejkę przerwań, która jest

obsługiwana w sprzętowej procedurze obsługi przerwań (*ISR*). W celu uniknięcia zjawiska wyścigu, pomiędzy ISR, a standardową pracą procesora, `InterruptService` stosuje mechanizmy synchronizacji, aby zapewnić bezpieczny dostęp do kolejki przerwań zarówno z poziomu ISR, jak i standardowej pracy procesora. W tym celu peyferium musi implementować interfejs `Interruptable`, aby możliwe było wyłączenie przerwań na czas modyfikacji kolejki.

> **WAZNE!** Po stronie użytkownika spoczywa odpowiedzialność na właściwe zarządzanie przerwaniami, poprzez metody interfejsu `Interruptable` peryferium którego dotyczy!

Do zarządzania `InterruptService` służy klasa `InterruptServiceTask`. Jest to specjalna implementacja klasycznego zadania `ServiceTask`, który wywołuje odpowiednie metody serwisu przerwaniowego, do zarządzania przerwaniami. W pierwszej kolejności obsługiwane są zgłoszone przerwania każdego serwisu. Dopiero potem są przetwarzane wiadomości.

> **WAZNE!** Zadanie `InterruptServiceTask` może subskrybować **TYLKO** serwisy `InterruptService`. Nie należy mieszać zwykłych serwisów z serwisami przerwaniowymi w ramach tego samego zadania!

3.4. Zadania specjalne

W module `dev` zaimplementowane zostały także taski do obsługi timera i schdulera. Task `Idle`oraz `Watchdog` służą do obsługi odpowiednio trybu bezczynności procesora oraz nadzorowania poprawności działania systemu, zdefiniowane w `nps::Scheduler`.

Task timera służy do wysyłania zdarzeń związanych z upływem określonego czasu, umożliwiając w ten sposób realizację operacji czasowych w aplikacjach użytkownika. Sposób korzystania z timera zdefiniowany jest w API modułu `dev`. Możliwe jest skonfigurowanie timera cyklicznie, aby generował zdarzenia w regularnych odstępach czasu. Aby task działał dokładnie, konieczne jest zapewnienie odpowiedniej precyzji zegara systemowego `hal::SystemTick` oraz wysokiego priorytetu taska timera.

4. System plików

Moduł **fs** biblioteki **npos** (katalog `npos/libnpos/fs`) definiuje interfejs systemu plików, poprzez wiadomości zdefiniowane w module `fs::msg`.

4.1. Wiadomości

Wiadomość	Znaczenie
file::Open	**otwarcie pliku z flagami (READ_ONLY, WRITE_ONLY, CREATE, TRUNCATE, APPEND, EXCLUSIVE)**
file::Read, file::Write	**odczyt i zapis do bufora użytkownika**
file::Close	**zamknięcie pliku**
dir::Open, dir::Close	**otwarcie i zamknięcie katalogu**
dir::Read	**odczyt kolejnego wpisu (typ, rozmiar, nazwa do 64 znaków)**
dir::Make	**utworzenie katalogu**

Otwarcie zwraca Oid pliku lub katalogu. Odczyt katalogu kończy status `END_OF_DIRECTORY`; wpisy `.` i `..` nie są zwracane. Otwarcie zbyt wielu plików lub katalogów w systemie skutkuje odpowiedzią `NO_MEMORY`.

4.2. Obsługa systemu plików

W pliku `libnpos/fs/Api.hpp` zdefiniowane są funkcje do obsługi systemu plików. Autor rekomenduje wywoływanie funkcji API do wysyłania żądań do systemu plików, co pozwoli na ewentualną zgodność aplikacji użytkownika w przypadku aktualizacji wiadomości w module `fs::msg`.

Przetwarzanie odpowiedzi wymaga sprawdzenia typu wiadomości (`ipc::Message::Type`), rzutowania jej na odpowiednią strukturę modułu `fs::msg`, a następnie skorzystania z getterów i setterów udostępnionych przez tą strukturę wiadomości. Istotnym elementem jest sprawdzenie statusu wiadomości, gdyż serwisy urządzeń jedynie zgłaszają błędy, ale to po stronie użytkownika spoczywa odpowiedzialność za ich obsłużenie w ramach pisanej aplikacji.

4.3. Implementacja systemu plików

Aktualnie implementacja systemu plików opiera się na pamięci flash i wykorzystuje bibliotekę littlefs do zarządzania strukturą katalogów i plików. System plików obsługuje podstawowe operacje na plikach i katalogach, takie jak otwieranie, odczyt, zapis, zamykanie oraz tworzenie katalogów.

Implementacja systemu plików zrealizowana jest w zadaniu `task::Littlefs`. OID pliku lub katalogu składa się z identyfikatora portu taska oraz lokalnego identyfikatora pliku lub katalogu w tablicy plików lub katalogów tego taska. Tworząc task należy podać rozmiar tablicy plików i katalogów, które będą obsługiwane przez ten task oraz kolejkę wiadomości

zadania.

4.3.1. Obsługa sprzętu

Pamięć opisuje `fs::hal::Flash` (blokowy dostęp do pamięci flash) i opcjonalnie `fs::hal::ICache` (pamięć podręczna instrukcji jest wyłączana na czas programowania). Jeżeli montowanie się nie powiedzie, zadanie formatuje pamięć i montuje ją ponownie.

4.3.2. Własna implementacja systemu plików

Istnieje możliwość implementacji systemu plików w oparciu o inną bibliotekę niż *littlefs*. Wymagane jest zachowanie spójności z wiadomościami modułu `fs::msg`. Możliwe jest także utworzenie w systemie więcej niż jednej instancji systemu plików (np. system plików na pamięci RAM i FLASH jednocześnie, lub kilka układów pamięci jednocześnie). Autor rekomenduje rozróżnianie instancji systemu poprzez ścieżki do katalogów głównych poszczególnych instancji (np. /flash/plik.txt i /ram/plik.txt). W ten sposób użytkownik może wysyłać wiadomości rozgłoszeniowe Open do wszystkich instancji, a odpowiadać będzie ten system plików, którego katalog główny odpowiada ścieżce podanej w żądaniu.

5. Terminal

Moduł **cli** biblioteki **npos** (katalog `npos/libnpos/cli`) zawiera terminal tekstowy oraz programy uruchamiane z jego linii poleceń.

5.1. Terminal

Zadanie `cli::task::Terminal` (wzorowane na zadaniu timera: zadanie, port i kolejka wiadomości) otwiera urządzenie wskazane nazwą i wysyła do niego `Transciever` w trybie odczytu na do 64 bajtów. Każdy odebrany bajt jest naciśnięciem klawisza. Wyjście jest kolejkowane i wysyłane porcjami, a gdy poprzedni zapis jeszcze trwa, dane czekają w buforze.

Przy uruchomieniu zadania (`initialize()`) terminal kolejkuje powitanie z wersją systemu z `npos::VERSION`, które trafia na wyjście zaraz po otwarciu urządzenia i przed pierwszym znakiem zachęty. Terminal nie czeka na podłączenie konsoli, bo interfejs `hal::UartPolling` i `hal::UartInterrupt` nie udostępnia informacji o połączeniu.

- Edycja linii: backspace, Delete, Home, End, strzałki lewo i prawo, Ctrl-A, Ctrl-E, Ctrl-U (kasuje linię), Ctrl-C (porzuca linię). Rozpoznawane są sekwencje escape CSI i SS3.
- Terminal prowadzi bufor linii i położenie kursora oraz robi lokalne echo.
- Historia pamięta pięć ostatnich poleceń, a strzałki w górę i w dół ją przełączają, przywracając linię rozpoczętą przed wejściem w historię.
- Log odebrany jako `cli::msg::SYSLOG` jest wypisywany nad edytowaną linią: terminal kasuje linię, wypisuje log w postaci `[{port:serwis} poziom] tekst` i rysuje linię ponownie wraz z kursorem. Wykrzykniki `!!!` oznaczają, że tekst został obcięty. Logi poniżej poziomu ustawionego przez `setLogLevel()` (domyślnie INFO) są pomijane.
- Terminal śledzi bieżący katalog (najwyżej 64 znaki) i pokazuje go w znaku zachęty.

Terminal jest własnym zadaniem i własnym portem na magistrali, a jednocześnie `Service::Api` swoich programów, więc dostaje wszystkie odpowiedzi do swoich żądań niezależnie od identyfikatorów serwisów w innych zadaniach. Odpowiedzi na cudze żądania są pomijane po porcie nadawcy.

Tekst logu leży w buforze nadawcy, który istnieje tylko w czasie wysyłania. Dlatego przy odbiorze terminal kopiuje go do slotu puli (`PayloadPoolInt`), podmienia `input.data` i dopiero wtedy kolejkuje wiadomość. Slot jest zwalniany po obsłużeniu logu. Gdy brakuje slotu, tekst jest za długi albo kolejka jest pełna, log jest odrzucany.

5.2. Logi

Logowanie należy do modułu cli i nie wymaga zmian w ipc. Serwis loguje przez `cli::api::syslog()` i strumień `cli::utils::LogStream`:

```
cli::api::syslog(*this, cli::log::WARNING) <<
```

Strumień składa tekst w buforze o stałym rozmiarze (MAX_MESSAGE_LENGTH = 128 znaków) i obsługuje napisy, liczby, wartości logiczne oraz bajty zapisywane szesnastkowo. Zbyt długi tekst ustawia flagę przepełnienia i dalsze dopiski są ignorowane. Dopiero `cli::log::endl` wysyła wiadomość `cli::msg::Syslog` (poziom, Oid nadawcy, flaga przepełnienia i tekst) przez `Api` serwisu. Wiadomość jest rozgłoszeniem, więc trafia do portów, które akceptują jej typ, z pominięciem portu nadawcy. Zapomniane `endl` oznacza, że wpis nie zostanie wysłany.

5.3. Programy

Program (`cli::Program`) jest serwisem. Terminal nie subskrybuje go do zadania, tylko przekazuje mu odpowiedzi działającego programu. Program implementuje ,`start(host,`args) odpowiedzi w `handleResponse()` i opcjonalnie `resume()`, wywoływane gdy wyjście terminala zostało opróżnione. Terminal udostępnia programom interfejs `Program::Host`: `print()`, `exit()` oraz odczyt i zmianę katalogu bieżącego. Wydruk zwraca liczbę przyjętych znaków, a program kontynuuje po `resume()`, dzięki czemu wypisanie dużego pliku nie wymaga dużego bufora.

Ścieżki względne są rozwiązywane względem katalogu bieżącego terminala funkcją `text::resolve()`, która obsługuje `.` i `..`.

Program	**Działanie**
cat <plik>	wypisuje plik
echo <tekst> [> | >> <plik>]	wypisuje tekst albo zapisuje g
touch <plik>	tworzy pusty plik
ls [katalog]	wypisuje wpisy katalogu
cd [katalog]	zmienia katalog, bez argumen do /
pwd	wypisuje katalog bieżący
mkdir <katalog>	tworzy katalog
random <rozmiar> [> | >> <plik>] T{ losuje bajty i wypisuje je szesnastkowo albo zapisuje surowe do pliku T}	
date [-s RRRR-MM-DD [GG:MM[:SS]]] T{ wypisuje czas urządzenia zegara albo go ustawia T}	

Program `random` ma generator, który zachowuje stan między uruchomieniami, więc każde wywołanie daje inne bajty. Ziarno jest stałe, więc po każdym uruchomieniu systemu ciąg jest ten sam. Program `date` otwiera zegar o nazwie `Config::clockDevice` (domyślnie `clock`) i wysyła `dev::msg::Clock`. Na otwarcie urządzenia, które nie istnieje, nikt nie odpowiada, więc program rejestruje timer i po 1000 tyknięciach kończy z błędem. Czas jest czasem UTC, a funkcje przeliczające znacznik czasu na datę i z powrotem są w `cli::utils::text`.

6. Emulator

Biblioteka **npos-emulator** zawiera implementacje HAL do uruchamiania systemu na komputerze: Flash, ICache, Led, Rtc, SystemTick, Hash, UartInterrupt, UartPolling, BackupRegisters, Power, Watchdog oraz ConsoleUart (stdin i stdout).

Mocki, które coś wypisują, mają argument konstruktora `verbose` (domyślnie wyłączony). Wydruki są włączane tylko w programach przykładowych, a testy jednostkowe są ciche.

ConsoleUart zwraca dane tak szybko, jak są dostępne, a resztę bufora wypełnia zerami, co terminal ignoruje.

7. Organizacja projektu, testy i przykłady

7.1. Biblioteki

Projekt składa się z trzech bibliotek budowanych z głównego `Makefile`:

npos
: rdzeń systemu w katalogu `npos/libnpos`, podzielony na moduły `nps`, `ipc`, `dev`, `fs` i `cli`; dołącza się je jako `<libnpos/moduł/Plik.hpp>`,

npos-emulator
: atrapy HAL do uruchamiania na komputerze,

Biblioteki są nagłówkowe, a zależności zewnętrzne to ETL (nagłówki) i littlefs. Moduły nie przydzielają pamięci. Kontenery, kolejki i pule podaje użytkownik.

7.2. Testy jednostkowe

Testy są napisane w CppUTest. Testy rdzenia są w `npos/tests`, w podkatalogach odpowiadających modułom (`nps`, `ipc`, `dev`, `fs`, `cli`), i tworzą jeden program testowy `test-npos`. Biblioteki `npos-emulator` i `npos-os` mają własne katalogi `tests`. Testy uruchamia Makefile z głównego katalogu przy budowaniu biblioteki, a niepowodzenie testu przerywa budowanie.

- Plik testowy modułu ma nazwę `Moduł.test.cpp` i leży w katalogu odpowiadającym modułowi.
- Pliki obiektowe są nazywane bez katalogu, a w rdzeniu jest teraz jeden program testowy, więc nazwa testu musi być unikalna w całej bibliotece. Gdy dwa moduły mają tę samą nazwę, do nazwy testu dodaje się przyrostek (na przykład `ApiDev.test.cpp`, `ApiFs.test.cpp`, `OpenDev.test.cpp`, `DirOpen.test.cpp`, `TimerTask.test.cpp`).
- Katalogi z testami są wymienione w `npos/tests/Makefile` jako kolejne wzorce `wildcard`. Nowy podkatalog z testami trzeba tam dopisać, w przeciwnym razie jego testy nie zostaną skompilowane.
- Wspólne atrapy (`RecordingApi`, `RecordingBus`, `FakeBus`, `RamFlash`, `Console` i inne) są w `npos/tests/mock`.
- Nagłówki projektu należy dołączać przed `CppUTest/TestHarness.h`, ponieważ ten nagłówek podmienia operator `new`.

7.3. Dokumentacja

Dokumentację buduje się poleceniem `make -C docs`. Zródłami są pliki rozdziałów `docs/source/NN_nazwa.roff` łączone w `main.roff`, a wynikiem jest `docs/manual.pdf`.

7.4. Przykłady

Katalog `examples` zawiera programy korzystające z `os::System` i emulatora: blinking-led, get-time, compute-hash, uart-read-write, filesystem, dual-system oraz cli (terminal na stdin i stdout, na przykład `printf 'ls\n' | ./bin/Pc32/debug/cli.elf`). Wskazówka: polecenia podawane przez potok należy rozdzielać pauzami, ponieważ terminal ignoruje bajty wpisane w trakcie działania programu.

Spis treści

1. Scheduler . . . 2
   1.1. Task . . . 2
2. Komunikacja międzyprocesowa . . . 4
   2.1. Adresowanie . . . 4
   2.2. Wiadomości . . . 4
      2.2.1. Typy wiadomości . . . 5
      2.2.2. Status wiadomości . . . 5
      2.2.3. Definiowanie własnych wiadomości . . . 5
   2.3. Serwisy . . . 6
      2.3.1. Maszyny stanów . . . 6
   2.4. ServiceTask . . . 7
   2.5. Magistrala systemowa . . . 8
      2.5.1. Porty i adresowanie . . . 8
   2.6. Przechowywanie wiadomości . . . 9
3. Urządzenia . . . 10
   3.1. Otwieranie urządzeń . . . 10
   3.2. Obsługa urządzeń . . . 10
   3.3. Obsługa przerwań . . . 10
   3.4. Zadania specjalne . . . 11
4. System plików . . . 12
   4.1. Wiadomości . . . 12
   4.2. Obsługa systemu plików . . . 12
   4.3. Implementacja systemu plików . . . 12
      4.3.1. Obsługa sprzętu . . . 13
      4.3.2. Własna implementacja systemu plików . . . 13
5. Terminal . . . 14
   5.1. Terminal . . . 14
   5.2. Logi . . . 14
   5.3. Programy . . . 15
6. Emulator . . . 17
7. Organizacja projektu, testy i przykłady . . . 19
   7.1. Biblioteki . . . 19
   7.2. Testy jednostkowe . . . 19
   7.3. Dokumentacja . . . 19
   7.4. Przykłady . . . 19